Amir Noori - Jesper Holmström - Mehdi Zaidane - Osama Aamer

Employee Management System

Metropolia University of Applied Sciences
Information and communication technologies
Software Engineering Project 1
Employee Management System
14.01.2026

Table of contents

1. Project Overview 1

    1.1. Project title 1
    1.2. Problem Summary 1
    1.3. Intended Audience/Users 1
    1.4. Main Features and Components 1

2. Project Objectives 1

3. Scope and Deliverables 2

    3.1. What Will Be Included 2
    3.2. What Will Be Excluded 2

4. Project Timeline 2

    4.1. Sprint 1 (Weeks 1–2) Project Initiation and Planning 3
    4.2. Sprint 2 (Weeks 3–4) Core Foundations and Initial Development 3
    4.3. Sprint 3 (Weeks 5–6) CI/CD Integration, Feature Expansion, and Docker 3
    4.4. Sprint 4 (Weeks 7–8) Finalization, Deployment, and Presentation 4

5. Resource Allocation 4

    5.1. Team members and roles 4
    5.2. Software and Tools 5

6. Risk Management 5

7. Testing and Quality Assurance 6

8. Documentation and Reporting 7

1. Project Overview

1.1. Project title

Employee Management System (EMS)

1.2. Problem Summary

Managing employee information using spreadsheets often leads to problems such as data loss, duplication, difficulty locating specific employees, and inefficiency when handling large amounts of information. As companies grow, spreadsheet-based management becomes increasingly complex and time-consuming.

The Employee Management System addresses these challenges by providing a structured, centralized, and user-friendly platform for managing employee data. The system improves accuracy, reduces manual work, and enhances productivity.

1.3. Intended Audience/Users

Organizations of all sizes, from small businesses to large companies, that need an efficient way to manage employees, departments, leave, and salaries.

1.4. Main Features and Components

- Five main views: Home, Employee, Department, Leave, and Salary
- Filter employees by department, position, and status
- Fast employee search by name
- Department-specific reports and insights
- Overview of all employees currently on leave
- Salary management and tracking for all employees

2. Project Objectives

The key objectives of this project are:

- Help companies manage employees efficiently and reliably
- Reduce time spent on manual searches and data handling
- Provide clear reports to improve departmental performance
- Track salaries accurately and calculate total monthly salary expenses

3. Scope and Deliverables

3.1. What Will Be Included

- A fully functional employee management application with five main modules
- Database integration for storing employee, department, leave, and salary data
- User-friendly interface built with JavaFX
- Search and filter functionality
- Department and salary reporting features

3.2. What Will Be Excluded

- Mobile application versions
- Multi-Language support
- Advanced analytics or machine learning features
- Integration with external payroll systems

4. Project Timeline

The project follows a four-sprint Agile structure aligned with the course schedule. Each sprint lasts two weeks and concludes with a Sprint Review session with the Product Owner. The timeline below outlines the phases, objectives, milestones, and dependencies for the Employee Management System.

4.1. Sprint 1 (Weeks 1–2) Project Initiation and Planning

Sprint 1 focuses on establishing the foundation of the project. The team forms roles, selects the project topic, and defines the product vision. During this sprint, the team prepares the initial project plan, sets up project management tools such as Trello or Jira, and creates the first version of the product backlog. The goal is to ensure that the team has a clear understanding of the project's scope, objectives, and workflow before development begins.

Dependencies: None, this sprint initiates the project.

Milestone: Submission of the Product Vision, Project Plan, and initial backlog.

4.2. Sprint 2 (Weeks 3–4) Core Foundations and Initial Development

The objective of Sprint 2 is to lay the technical foundation for the application. The team implements the database schema and creates the initial MariaDB tables. Early user interface development begins using JavaFX, focusing on basic navigation and layout. Essential development tools are integrated, including Maven dependencies, unit testing frameworks, and code coverage reporting. By the end of this sprint, the system should have a functional skeleton with working database connectivity.

Dependencies: Completion of Sprint 1 planning and backlog creation.

Milestone: Functional database, initial UI screens, and working development environment.

4.3. Sprint 3 (Weeks 5–6) CI/CD Integration, Feature Expansion, and Docker

Sprint 3 expands the functional prototype and introduces automation into the development workflow. The team integrates Jenkins to establish a CI/CD pipeline, enabling continuous integration and automated builds. Additional features are implemented to strengthen backend logic and improve system functionality. Automated testing is enhanced with code coverage tools to ensure software quality. The team also begins preparing a local Docker image of the application as a step toward deployment.

Dependencies: Core modules and database integration from Sprint 2 must be stable.
Milestone: Working CI/CD pipeline, extended features, and first Docker image.

4.4. Sprint 4 (Weeks 7–8) Finalization, Deployment, and Presentation

The final sprint focuses on completing the system's functionality and preparing the project for deployment and demonstration. The application is fully containerized using Docker, and the Docker image is pushed to Docker Hub. The team tests the deployed image in a suitable environment to ensure reliability. The system is shared with classmates for feedback, and final adjustments are made based on review comments. The sprint concludes with the final demonstration and submission of all deliverables.

Dependencies: CI/CD pipeline and functional prototype from Sprint 3.

Milestone: Final demonstration, Docker deployment, and submission of all project materials.

Trello Link: Project Timeline

5. Resource Allocation

The project team follows the Scrum framework, with clearly defined roles and responsibilities. To ensure equal participation and familiarity with Agile practices, the Scrum Master role rotates at the end of each sprint.

5.1. Team members and roles

Product Owner Lecturer Amir Dirin:

The Product Owner is responsible for defining feature priorities, clarifying requirements, and reviewing the team's progress at the end of each sprint. The Product Owner ensures that the project remains aligned with the course objectives and the product vision.

Scrum Master Amir Noori:

During Sprint 1, Amir Noori serves as the Scrum Master. His responsibilities include facilitating team meetings, organizing the sprint planning session, ensuring that Agile principles are followed, and coordinating the submission of Sprint Review materials. At

the end of Sprint 1, the Scrum Master role will rotate to the next team member. Each sprint will have a different Scrum Master so that all team members gain experience in leading Agile ceremonies and supporting team collaboration.

Development Team:

Our development team members are Amir Noori, Osama Aamer, Mehdi Zaidane, Jesper Holmström The development team is responsible for implementing the system's functionality, including JavaFX development, database integration, testing, and DevOps setup. Team members collaborate closely throughout all sprints, contributing to coding, design, documentation, and review activities.

5.2. Software and Tools

The project uses a variety of tools to support development, collaboration, and deployment:

- Project Management: Jira for Tracking Sprint's Backlog and user stories
- Modeling: StarUML(Version 5.0.1) for creating Use Case diagrams
- Build Automation: Maven (Required in Sprint 2)
- CI/CD: Jenkins (Required for Sprint 3)
- Development: Docker (Required for Sprint 4)
- Version Control: GitHub

6. Risk Management

Potential Risks that could impact the success of the Employee Management System.

Risk Description	Likelihood	Impact	Mitigation Strategies
Team Constraints: 49% of projects fail to finish on time	High	High	Adhering strictly to the Agile sprints. If a feature is not ready, we move it to the backlog rather than delaying the release

Scope Creep: Adding unnecessary features (example: complex external payroll integration)	Medium	High	Sticking to Use Case Diagrams boundaries, & only implementing "key features" listed in the Product Vision
Technical Debt: Poor Quality code leading to bugs later down the road	Medium	Medium	Contact regular code reviews and writing tests during each phase, especially in the "implementation phase"
Data Integrity Issues: Errors in salary calculation or data loss	low	High	Implementing regular testing for relevant modules such as "Salaries" module and ensuring the database backups are functional

7. Testing and Quality Assurance

The Employee Management System will follow a structured and systematic approach to testing and quality assurance to ensure that the final product is reliable, functional, and aligned with the project requirements. Testing activities will begin early in the development process and continue throughout all sprints. The primary focus will be on verifying that each component of the system behaves as expected and integrates smoothly with other parts of the application.

Unit testing will be conducted using JUnit to validate the correctness of individual methods and classes, particularly those responsible for business logic and database

interactions. Integration testing will be performed to ensure that the JavaFX interface, JDBC layer, and MariaDB database operate together without errors. As features are completed, user acceptance testing will be carried out to confirm that the system meets the expectations of the intended users, including HR administrators, managers, and employees. Regression testing will be performed at the end of each sprint to ensure that newly added features do not negatively affect existing functionality.

Quality assurance will also involve continuous monitoring of code quality through tools such as Jenkins and JaCoCo, which will help maintain consistent coding standards and track code coverage. The success of the testing process will be measured by the stability of the system, the absence of critical defects, and the ability of users to perform their tasks without encountering errors. The goal is to deliver a robust and user-friendly application that meets all functional and non-functional requirements.

8. Documentation and Reporting

Comprehensive documentation will be produced throughout the development of the Employee Management System to support both the development team and the end users. The documentation will include technical materials such as UML diagrams, database schema descriptions, architectural explanations, and implementation details. These documents will help ensure that the system is maintainable, understandable, and aligned with the project's design principles. In addition to technical documentation, user-oriented materials such as a user manual and usage instructions will be prepared to guide HR administrators, managers, and employees in operating the system effectively.

The Use Case Diagram in Figure 1 illustrates the main functional interactions between the user and the Employee Management System. The system has a single primary actor, the Manager, who has full control over all system functionalities. The Manager must authenticate using the Login use case before accessing the system. Once authenticated, the Manager is responsible for all administrative tasks, including managing employees, managing departments, handling employee leave requests, and managing salaries. This diagram provides a high-level overview of the system's behavior and clearly defines the scope of the Manager's responsibilities and the overall system boundaries.

"The HR Admin role is conceptually merged into the Manager role in the project scope."

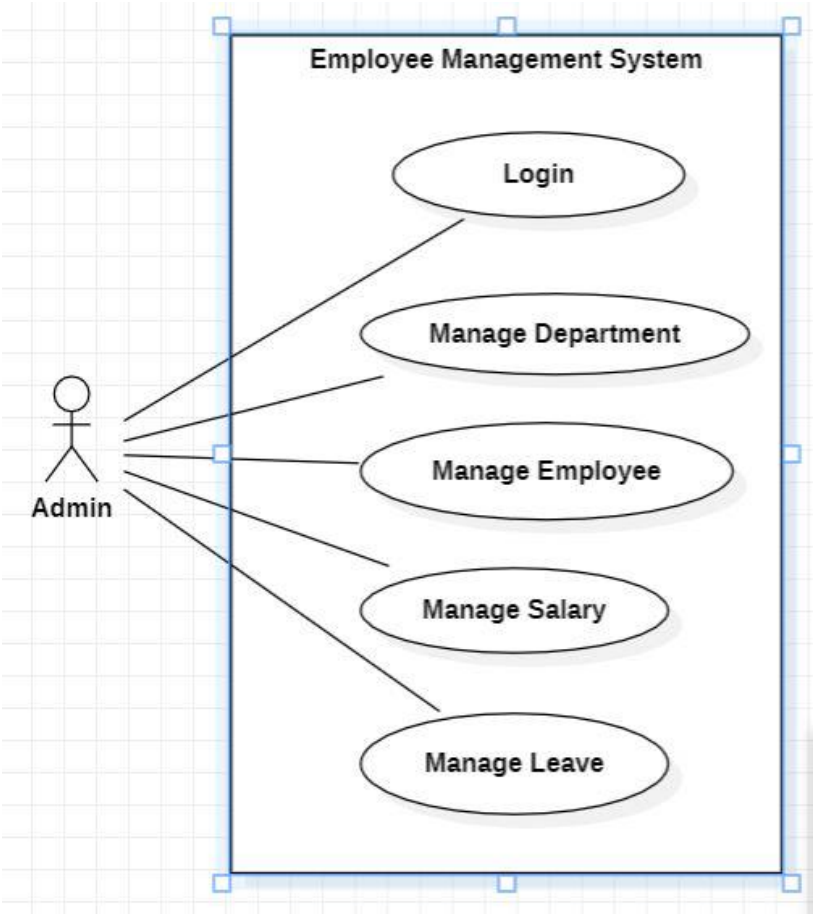


Figure 1: Use Case Diagram of the Employee Management System

Project reporting will occur on a regular basis to track progress and maintain transparency within the team. Weekly meetings will be held to review completed tasks, identify challenges, and plan upcoming work. At the end of each sprint, a sprint review will be conducted to demonstrate the implemented features and evaluate the team's progress against the sprint goals. Updates will also be recorded in the team's project management tool, such as Jira or Trello, to maintain an organized and accessible record of the project's development. The final stages of the project will include the preparation of a comprehensive final report and a presentation summarizing the system's functionality, design, and implementation.